МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ
Ордена Трудового Красного Знамени
Федеральное государственное бюджетное образовательное учреждение высшего образования
«Московский технический университет связи и информатики»

Кафедра «Программная инженерия»

Лабораторная работа №3

по дисциплине «Структуры и алгоритмы обработки данных»
по теме «Анализ графов и кратчайшие пути»

Выполнил студент группы:
БПИ2401
Еланская Екатерина Дмитриевна
Проверил: Борзов В.М.

Москва, 2026

Оглавление

Введение **3**

Выполнение **4**
Задание 1. Представление графа и базовый обход 4
Задание 2. Компоненты сильной связности 5
Задание 3. Мосты и точки сочленения 6
Задание 4. Граф конденсации, топологическая сортировка и цикл 6
Задание 5. Алгоритм Дейкстры и восстановление маршрута . . 6
Задание 6. Беллман–Форд и отрицательный цикл 7
Задание 7. Флойд–Уоршелл и итоговое сравнение 7

Вывод **8**

Контрольные вопросы **9**

Введение

Целью данной лабораторной работы является:

— изучить методы анализа структуры ориентированных и неориентированных графов;

— освоить поиск компонент сильной связности, мостов, точек сочленения и топологическую сортировку;

— реализовать и сравнить основные алгоритмы поиска кратчайших путей во взвешенных графах.

Задачи:

1) научиться представлять граф списком смежности;

2) научиться применять DFS как основу для структурного анализа;

3) научиться использовать времена входа tin и значения low;

4) реализовать алгоритм Дейкстры с приоритетной очередью, алгоритм Беллмана–Форда и алгоритм Флойда–Уоршелла;

5) научиться определять условия применимости каждого метода.

Выполнение

Исходные данные графа G представлены в листинге 1.

```
V = {0, 1, 2, 3, 4, 5, 6, 7}

edges = [
    (0, 1, 4),
    (0, 2, 8),
    (1, 2, 1),
    (2, 0, 3),
    (2, 3, 2),
    (3, 4, 2),
    (4, 5, 3),
    (5, 3, 1),
    (5, 6, 4),
    (6, 7, 2),
]
```

Листинг 1 — Исходные данные графа G

Задание 1. Представление графа и базовый обход

Код по заданию 1 представлен в листинге 2.

```
def build_directed(edges):
    G = {v: [] for v in V}
    for u, v, w in edges:
        G[u].append((v, w))
    return G

def build_undirected(edges):
    Gu = {v: [] for v in V}
    seen = set()
    for u, v, w in edges:
        e = (min(u, v), max(u, v))
        if e in seen:
            continue
        seen.add(e)
        Gu[u].append(v)
        Gu[v].append(u)
    return Gu
def dfs(Gu, start=0, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    order = [start]
    for u in Gu[start]:
        if u not in visited:
            order += dfs(Gu, u, visited)
    return order

def components(Gu):
    visited = set()
    comps = []
    for v in sorted(V):
        if v not in visited:
            tmp = dfs(Gu, v, visited)
            comps.append(sorted(tmp))
    return comps
```

Листинг 2 — Построение списков смежности и обход DFS для G_u

При представлении списком смежности каждая вершина хранится один раз, и для нее хранится список ее соседей. Обход графа и помечение каждой посещенной вершины равно O(V). Просмотр всех элементов списков смежности для каждой вершины – O(E). Таким образом, общая сложность равна O(V) + O(E) = O(V + E).

Задание 2. Компоненты сильной связности

Код по заданию 2 представлен в листинге 3.

```
def kosaraju(G):
    def dfs1(v):
        visited.add(v)
        for u, i in G[v]:
            if u not in visited:
                dfs1(u)
        order.append(v)

    def dfs2(v, comps, Gt):
        visited.add(v)
        comps.append(v)
        for u in Gt[v]:
            if u not in visited:
                dfs2(u, comps, Gt)

    visited = set()
    order = []
    for v in sorted(V):
        if v not in visited:
            dfs1(v)

    Gt = {v: [] for v in G}
    for u in G:
        for e, i in G[v]:
            Gt[e].append(u)

    visited = set()
    sccs = []
    for v in reversed(order):
        if v not in visited:
            comp = []
            dfs2(v, comp, Gt)
            sccs.append(sorted(comp))
    return sccs
```

Листинг 3 — Алгоритм Косарайю для поиска компонент сильной связности

Компонента сильной связности требует, чтобы для любых двух вершин u и v существовал путь (u, v) и путь (v, u). У ребра {6, 7} есть только путь из 6 в 7.

Задание 3. Мосты и точки сочленения

Код по заданию 3 представлен в листинге 4.

```
timer = 0
tin, low = {}, {}
bridges, points = set(), set()

def dfs_bridges(v, parent, Gu):
    global timer
    tin[v] = low[v] = timer
    timer += 1
    child_count = 0
    for u in Gu:
        if u == parent:
            continue
        if u in tin:
            low[v] = min(low[v], low[u])
        else:
            child_count += 1
            dfs_bridges(u, v, Gu)
            low[u] = min(low[v], low[u])
            if low[u] > tin[v]:
                bridges.add(
                    (min(v, u), max(v, u))
                )
            if parent is not None and low[u] >= tin[v]:
                points.add(v)
    if parent is None and child_count > 1:
        points.add(v)

for v in sorted(V):
    if v not in tin:
        dfs_bridges(v, None, Gu)
```

Листинг 4 — Поиск мостов и точек сочленения с использованием tin и low

Задание 4. Граф конденсации, топологическая сортировка и цикл

Код по заданию 4 представлен в листинге 5.

```
def topology_sort():
```

Листинг 5 — Граф конденсации и топологическая сортировка

Задание 5. Алгоритм Дейкстры и восстановление маршрута

Код по заданию 5 представлен в листинге 6.

```
import heapq as hp
```

Листинг 6 — Алгоритм Дейкстры с приоритетной очередью и восстановление маршрута

Задание 6. Беллман–Форд и отрицательный цикл

Код по заданию 6 представлен в листинге 7.

```
V = {0, 1, 2, 3, 4, 5, 6, 7}
```

Листинг 7 — Алгоритм Беллмана–Форда и обнаружение отрицательного цикла

Задание 7. Флойд–Уоршелл и итоговое сравнение

Код по заданию 7 представлен в листинге 8.

```
V = {0, 1, 2, 3, 4, 5, 6, 7}
```

Листинг 8 — Алгоритм Флойда–Уоршелла и сравнение алгоритмов

Результат сравнения алгоритмов, рассмотренных в работе, представлен в таблице 1.

Таблица 1 — Сравнение алгоритмов Дейкстры, Беллмана–Форда и Флойда–Уоршелла

Алгоритм	**Допустимость отрицательных ребер**	**Возможность обнаружения отрицательного цикла**	**Асимптотическая сложность**
Дейкстра			
Беллман–Форд			
Флойд–Уоршелл			

Для одного источника без отрицательных весов следует выбирать алгоритм Дейкстры, одного источника с отрицательными ребрами – алгоритм Беллмана–Форда, для поиска расстояний между всеми парами вершин в графе умеренного размера – алгоритм Флойда-Уоршелла.

Вывод

В ходе лабораторной работы были получены:

— знания о том, как представлять граф списком смежности;

— знания о том, как применять DFS как основу для структурного анализа;

— знания о том, как использовать времена входа tin и значения low;

— реализации алгоритма Дейкстры с приоритетной очередью, алгоритма Беллмана–Форда и алгоритма Флойда–Уоршелла;

— знания о том, как определять условия применимости каждого метода.

Контрольные вопросы

1) *Что такое список смежности и какова стоимость обхода графа при таком представлении?*
   Список смежности – структура вида вершина: [соседи], где для каждой вершины хранится список смежных с ней вершин (и, при необходимости, весов ребер). Обход (DFS или BFS) стоит O(V + E): каждая вершина обрабатывается один раз, а каждый элемент списков смежности просматривается один раз (для неориентированного графа – дважды).

2) *Чем компонента сильной связности отличается от обычной компоненты связности?*
   Компонента связности (в неориентированном графе) – максимальное множество вершин, между любыми двумя из которых существует путь (достаточно пути в одну сторону). Компонента сильной связности (в ориентированном графе) – максимальное множество вершин, в котором для любых u, v существует путь (u, v) и путь (v, u).

3) *Из каких основных этапов состоит алгоритм Косарайю?*
   Алгоритм Косарайю состоит из трех этапов: а) DFS по исходному графу с сохранением вершин в порядке завершения обработки (post-order); б) построение транспонированного графа; в) DFS по графу из предыдущего пункта, запускаемый в обратном порядке относительно выхода из первого DFS.

4) *Какую роль играют tin и low при DFS?*
   tin[v] – время первого посещения вершины v при DFS (счетчик увеличивается на каждом входе). low[v] – минимальное значение tin, достижимое из v и ее DFS-поддерева с использованием ребер дерева и не более одного обратного ребра. Эти величины позволяют находить мосты и точки сочленения за один обход.

5) *Как определяется мост через значения tin и low?*
   Мост – ребро дерева DFS (v, to), для которого low[to] > tin[v].

6) *Что называется точкой сочленения и чем отличается обработка корня DFS?*
   Точка сочленения – вершина, удаление которой вместе с инцидентными ребрами увеличивает число компонент связности.

7) *Что такое DAG?*
   DAG (Directed Acyclic Graph) – ориентированный граф без ориентированных циклов.

8) *Что означает топологический порядок и когда он существует?*
Топологический порядок – линейное упорядочение вершин, при котором для каждого ребра (u, v) вершина u стоит раньше v. Он существует тогда и только тогда, когда граф является DAG (не содержит циклов).

9) *Как с помощью DFS обнаружить цикл в ориентированном графе?*
При DFS вершины раскрашиваются в три состояния: 0 – не посещена, 1 – в текущем стеке DFS, 2 – обработана. Если из текущей вершины обнаружено ребро в вершину со состоянием 1, это обратное ребро, то цикл есть.

10) *Что называется релаксацией ребра?*
Релаксация ребра (u, v) с весом w – операция вида:

```
if dist[v] > dist[u] + w:
    dist[v] = dist[u] + w
    parent[v] = u
```

Она пытается улучшить известное расстояние до v, используя путь через u.

11) *При каких условиях корректен алгоритм Дейкстры?*
При неотрицательных весах ребер.

12) *Зачем в Дейкстре используется приоритетная очередь?*
Приоритетная очередь в алгоритме Дейкстры нужна для эффективного выбора вершины с минимальным текущим расстоянием.

13) *Как восстановить маршрут после поиска кратчайших расстояний?*
При каждой успешной релаксации сохраняется parent[v] = u. После завершения алгоритма начинается от конечной вершины по parent до стартовой, во время которого собираются вершины. Полученный список затем разворачивается.

14) *Почему отрицательные рёбра создают проблему для алгоритма Дейкстры?*
Отрицательные ребра ломают жадность: вершина, расстояние до которой уже зафиксировано как минимальное, позже может получить более короткий путь через отрицательное ребро.

15) *Как работает алгоритм Беллмана–Форда?*
Беллман–Форд выполняет V-1 полных проходов по всем ребрам, на каждом проходе ослабляя все ребра. Если на очередном проходе не было ни одной релаксации, алгоритм завершается досрочно.

16) *Почему для кратчайшего пути без отрицательных циклов достаточно V-1 итераций релаксации?*
Достаточно V-1 итераций, потому что любой простой путь (без повторных вершин) из V вершин содержит не более V-1 ребер.

17) *Как Беллман–Форд обнаруживает отрицательный цикл?*
После V-1 итераций выполняется еще один проход по всем ребрам. Если хотя бы одно ребро можно ослабить (dist[u] + w < dist[v]), то существует цикл отрицательного веса, достижимый из старта, и конечного кратчайшего расстояния для некоторых вершин не существует.

18) *Какую задачу решает алгоритм Флойда–Уоршелла?*
Алгоритм Флойд–Уоршелла решает задачу поиска кратчайших расстояний между всеми парами вершин. На шаге k проверяется, можно ли улучшить путь (i, j), разрешив использовать вершину k как промежуточную.

19) *Какова временная сложность Дейкстры с двоичной кучей, Беллмана–Форда и Флойда–Уоршелла?*
Сложность Дейкстры с двоичной кучей – $O((V + E) \log V)$; Беллмана–Форда – $O(VE)$; Флойда–Уоршелла – $O(V^3)$ (память $O(V^2)$).

20) *В каких случаях следует выбирать каждый из трёх алгоритмов кратчайших путей?*
Для одного источника без отрицательных весов – алгоритм Дейкстры; для одного источника с возможными отрицательными ребрами – Беллмана–Форда; для поиска расстояний между всеми парами в графе умеренного размера – Флойда–Уоршелла. Для больших разреженных графов выгоднее Дейкстра из каждой вершины.